

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define the path to the data folder
data_folder = 'C:/Users/moses_y/OneDrive/Desktop/plots/data/'

# Load the GTFS files into DataFrames
agency = pd.read_csv(data_folder + 'agency.txt')
calendar = pd.read_csv(data_folder + 'calendar.txt')
calendar_dates = pd.read_csv(data_folder + 'calendar_dates.txt')
feed_info = pd.read_csv(data_folder + 'feed_info.txt')
frequencies = pd.read_csv(data_folder + 'frequencies.txt')
routes = pd.read_csv(data_folder + 'routes.txt')
shapes = pd.read_csv(data_folder + 'shapes.txt')
stop_times = pd.read_csv(data_folder + 'stop_times.txt')
stops = pd.read_csv(data_folder + 'stops.txt')
trips = pd.read_csv(data_folder + 'trips.txt')

# Explore the data and perform necessary data manipulations
# For example, you can print the head of a DataFrame to check the data
routes.head()
```

Out[]:

	route_id	agency_id	route_short_name	route_long_name	route_desc	route_t
0	10100011A11	UON	11A	Highridge	Odeon-AgaKhan-HighRidge	
1	10300011F11	UON	11F	Ruaka	Odeon-UN-Ruaka	
2	10300106011	UON	106	Banana	Odeon-UN-Ruaka-Banana	
3	10400010711	UON	107	Ndenderu	Odeon-UN-Ruaka-Ndenderu	
4	10200010811	UON	108	Gachie	UN-NewMuthaiga-Gachie-Gichagi	

trips.head()

Out []:

	route_id	service_id	trip_id	trip_headsign	direction_id	block_id	shape_id
0	10100011A11	DAILY	1011A111	From Odeon	1	NaN	1011A11
1	10100011A11	DAILY	1011A110	To Odeon	0	NaN	1011A11
2	10300011F11	DAILY	1011F111	From Odeon	1	NaN	1011F11
3	10300011F11	DAILY	1011F110	To Odeon	0	NaN	1011F11
4	10300106011	DAILY	10106111	From Koja	1	NaN	1010611

In []:

```
stops.head()
```

Out []:

	stop_id	stop_code	stop_name	stop_desc	stop_lat	stop_lon	zone_id	stop_u
0	0005AMB	NaN	Ambassadeur	D	-1.285963	36.826048	NaN	Na
1	0006BSS	NaN	Bus Station	D	-1.287191	36.828881	NaN	Na
2	0007COM	NaN	Commercial	D	-1.284282	36.826188	NaN	Na
3	0512BST	NaN	Bus Station	D	-1.287281	36.828863	NaN	Na
4	0512BSN	NaN	Bus Station	D	-1.287635	36.829215	NaN	Na

In []:

```
stop_times.head()
```

Out []:

	trip_id	arrival_time	departure_time	stop_id	stop_sequence	stop_headsign	picl
0	40006111	6:00:00	6:00:00	0412CAR	1	NaN	
1	40006111	6:02:00	6:02:00	0411RIK	2	NaN	
2	40006111	6:04:00	6:04:00	0411JNV	3	NaN	
3	40006111	6:06:00	6:06:00	0401BFR	4	NaN	
4	40006111	6:08:00	6:08:00	0411BFR	5	NaN	

In []:

```
shapes.head()
```

Out []:

	shape_id	shape_pt_lat	shape_pt_lon	shape_pt_sequence	shape_dist_traveled
0	4000611	-1.294705	36.853271	1001	NaN
1	4000611	-1.294484	36.852337	1002	NaN
2	4000611	-1.294484	36.852337	1003	NaN
3	4000611	-1.294378	36.851830	1004	NaN
4	4000611	-1.294322	36.851715	1005	NaN

```
In [ ]: calendar.head()
```

Out[]:

	service_id	monday	tuesday	wednesday	thursday	friday	saturday	sunday	start_
0	DAILY	1	1	1	1	1	1	1	2014
1	SPCDY	1	1	1	1	1	1	1	2014

```
In [ ]: trips.head()
```

Out[]:

	route_id	service_id	trip_id	trip_headsign	direction_id	block_id	shape_id
0	10100011A11	DAILY	1011A111	From Odeon	1	NaN	1011A11
1	10100011A11	DAILY	1011A110	To Odeon	0	NaN	1011A11
2	10300011F11	DAILY	1011F111	From Odeon	1	NaN	1011F11
3	10300011F11	DAILY	1011F110	To Odeon	0	NaN	1011F11
4	10300106011	DAILY	10106111	From Koja	1	NaN	1010611

```
In [ ]: frequencies.head()
```

Out[]:

	trip_id	start_time	end_time	headway_secs
0	1011A111	6:00:00	9:00:00	300
1	1011A111	9:01:00	15:00:00	900
2	1011A111	15:01:00	21:00:00	300
3	1011A110	6:00:00	9:00:00	300
4	1011A110	9:01:00	15:00:00	900

```
In [ ]: agency.head()
```

Out[]:

	agency_id	agency_name	agency_url	agency_timezone	agency_lang	agen
0	UON	University Of Nairobi C4DLab	http://www.uonbi.ac.ke	Africa/Nairobi	en	

```
In [ ]: calendar_dates.head()
```

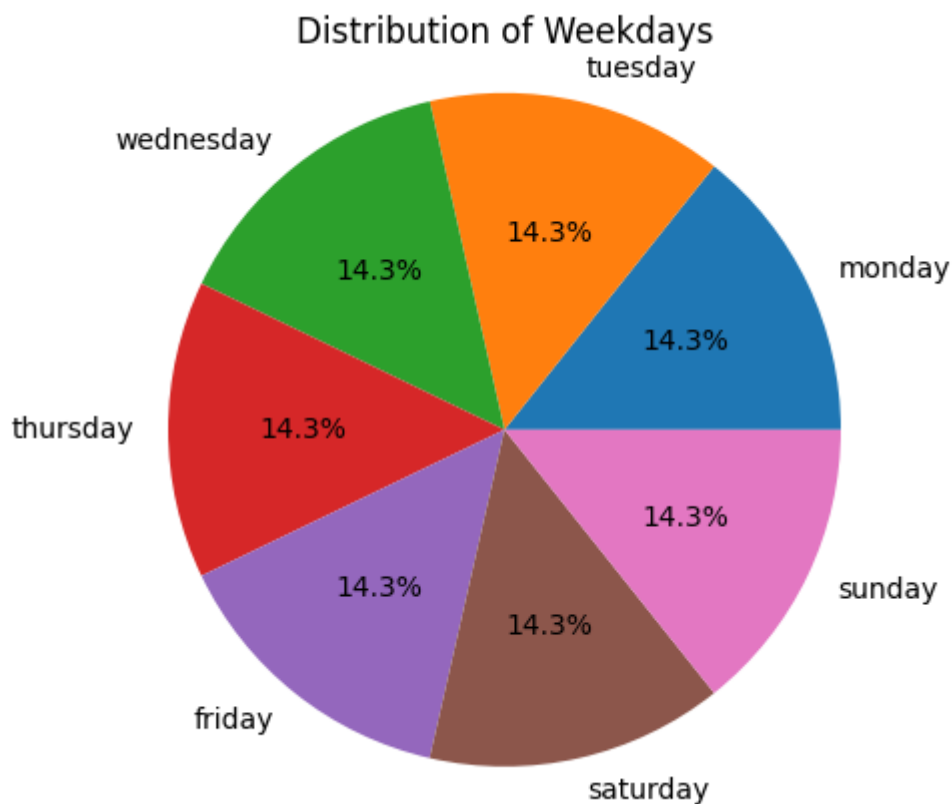
```
Out[ ]:
  service_id  date  exception_type
0    DAILY  20140526             2
1    SPCDY  20140310             2
```

```
In [ ]: feed_info.head()
```

```
Out[ ]:
  feed_publisher_name  feed_publisher_url  feed_lang  feed_start_date  feed_
0    Digital Matatus  http://www.digitalmatatus.com/    en    20120302
```

```
In [ ]: # Plot Pie Chart
weekday_columns = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday']
weekday_counts = calendar[weekday_columns].sum()

plt.pie(weekday_counts, labels=weekday_columns, autopct='%1.1f%%')
plt.title('Distribution of Weekdays')
plt.axis('equal')
plt.show()
```



```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define the path to the data folder
data_folder = 'C:/Users/moses_y/OneDrive/Desktop/plots/data/'

# Load the GTFS files into DataFrames
agency = pd.read_csv(data_folder + 'agency.txt')
calendar = pd.read_csv(data_folder + 'calendar.txt')
```

```
calendar_dates = pd.read_csv(data_folder + 'calendar_dates.txt')
feed_info = pd.read_csv(data_folder + 'feed_info.txt')
frequencies = pd.read_csv(data_folder + 'frequencies.txt')
routes = pd.read_csv(data_folder + 'routes.txt')
shapes = pd.read_csv(data_folder + 'shapes.txt')
stops = pd.read_csv(data_folder + 'stops.txt')
trips = pd.read_csv(data_folder + 'trips.txt')

# Select the necessary columns from the stops and trips DataFrames
stops_data = stops[['stop_id', 'stop_lat', 'stop_lon']]
trips_data = trips[['route_id', 'trip_id']]

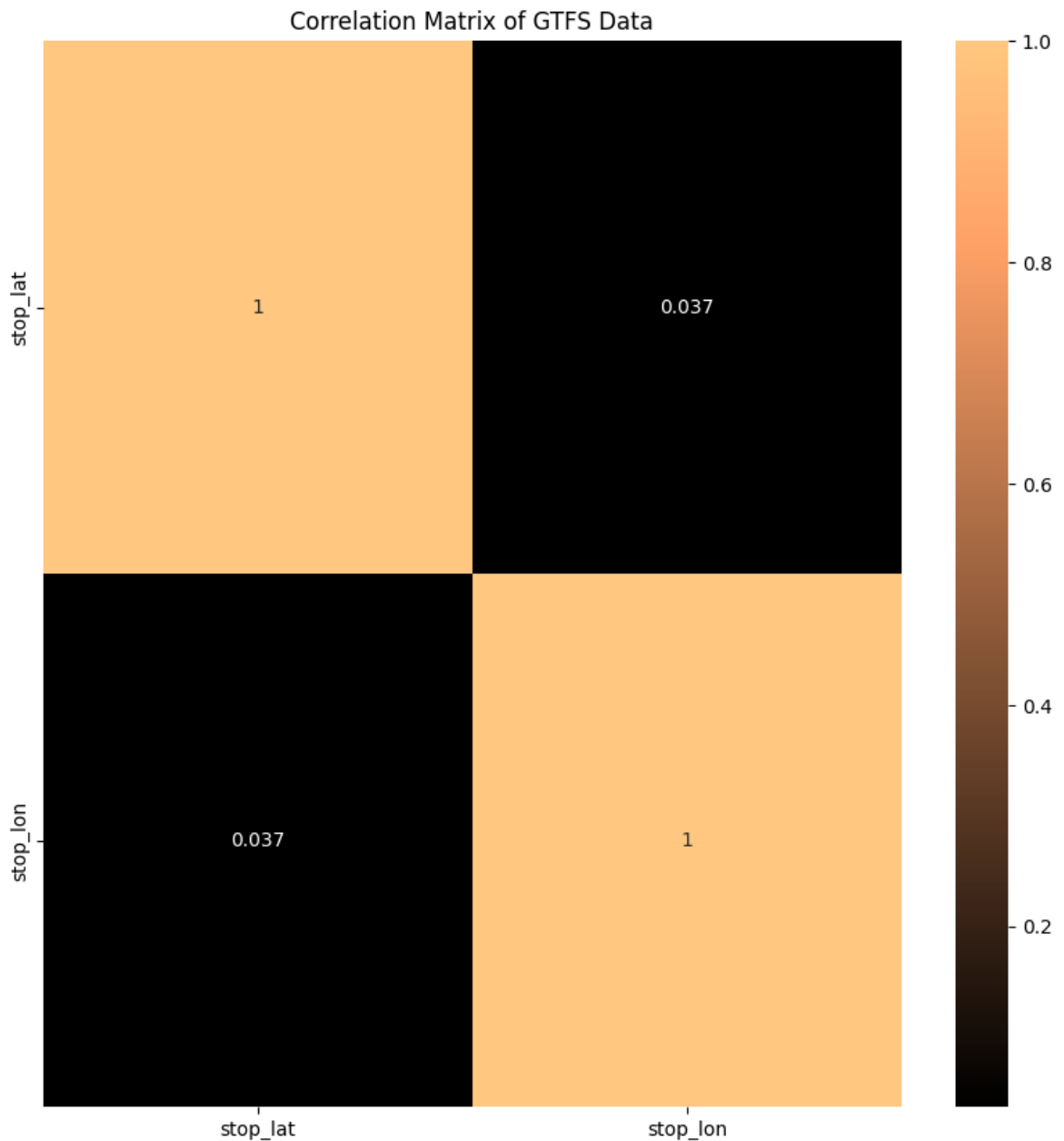
# Combine the selected DataFrames into a single DataFrame
all_data = pd.concat([stops_data, trips_data], axis=1)

# Exclude non-numeric columns from the combined DataFrame
numeric_columns = all_data.select_dtypes(include=[float, int]).columns
numeric_data = all_data[numeric_columns]

# Fill null values with the mean of respective columns
numeric_data_filled = numeric_data.fillna(numeric_data.mean())

# Calculate the correlation matrix
correlation_matrix = numeric_data_filled.corr()

# Plot the correlation matrix as a heatmap with adjusted color scheme
plt.figure(figsize=(10, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='copper')
plt.title('Correlation Matrix of GTFS Data')
plt.show()
```



```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define the path to the data folder
data_folder = 'C:/Users/moses_y/OneDrive/Desktop/plots/data/'

# Load the GTFS files into DataFrames
agency = pd.read_csv(data_folder + 'agency.txt')
calendar = pd.read_csv(data_folder + 'calendar.txt')
calendar_dates = pd.read_csv(data_folder + 'calendar_dates.txt')
feed_info = pd.read_csv(data_folder + 'feed_info.txt')
frequencies = pd.read_csv(data_folder + 'frequencies.txt')
routes = pd.read_csv(data_folder + 'routes.txt')
shapes = pd.read_csv(data_folder + 'shapes.txt')
stops = pd.read_csv(data_folder + 'stops.txt')
trips = pd.read_csv(data_folder + 'trips.txt')

# Merge 'trips' and 'frequencies' DataFrames on 'trip_id'
trips = trips.merge(frequencies, on='trip_id')
```

```

# Convert 'start_time' and 'end_time' columns to datetime type
trips['start_time'] = pd.to_datetime(trips['start_time'], format='%H:%M:%S')
trips['end_time'] = pd.to_datetime(trips['end_time'], format='%H:%M:%S')

# Check if 'trip_duration' column exists in the DataFrame
if 'trip_duration' not in trips.columns:
    # Calculate trip duration
    trips['trip_duration'] = trips['end_time'] - trips['start_time']

# Convert trip_duration to numeric format
trips['trip_duration'] = trips['trip_duration'].dt.total_seconds() / 60

# Filter out invalid or missing data
trips = trips.dropna(subset=['route_id', 'trip_duration'])

# Check if any valid trip durations exist
if trips['trip_duration'].empty:
    print("No valid trip durations found.")
else:
    # Calculate average trip duration
    average_trip_duration = trips['trip_duration'].mean()
    print("Average Trip Duration:", average_trip_duration)

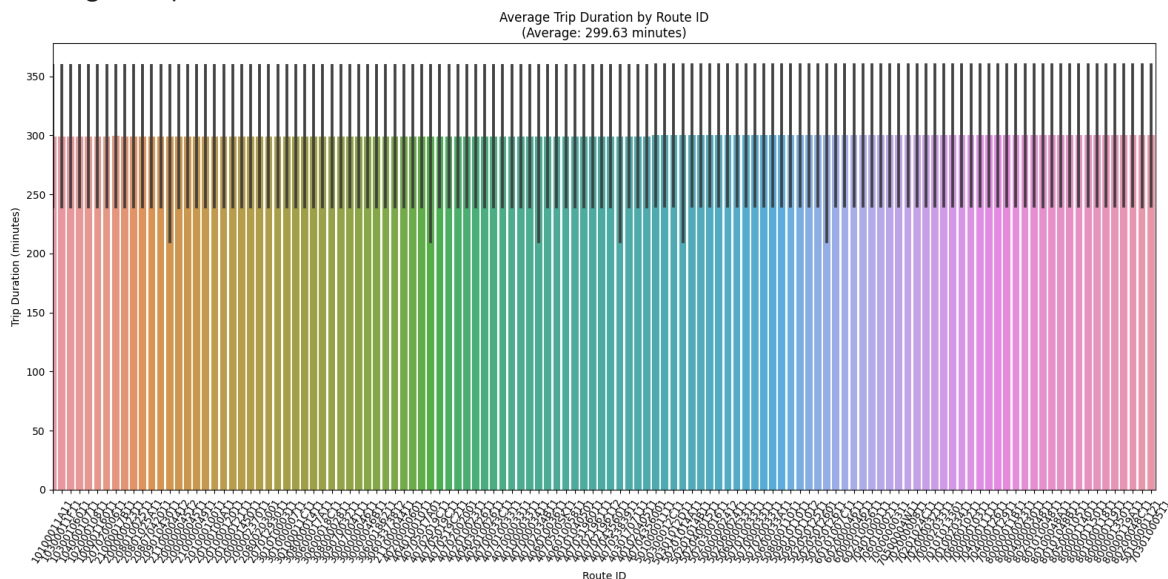
    # Plot the bar chart for average trip duration by route ID
    plt.figure(figsize=(16, 8)) # Adjust the figure size
    ax = sns.barplot(x='route_id', y='trip_duration', data=trips)
    plt.xlabel('Route ID')
    plt.ylabel('Trip Duration (minutes)')
    plt.title(f'Average Trip Duration by Route ID\n(Average: {average_trip_duration})')
    plt.xticks(rotation=60) # Rotate the x-axis labels by 30 degrees
    plt.tight_layout() # Adjust the spacing of the plot

    # Adjust the x-axis labels to start at 0
    ax.set_xlim(left=0)
    plt.ylim(0, None)

plt.show()

```

Average Trip Duration: 299.63315266486

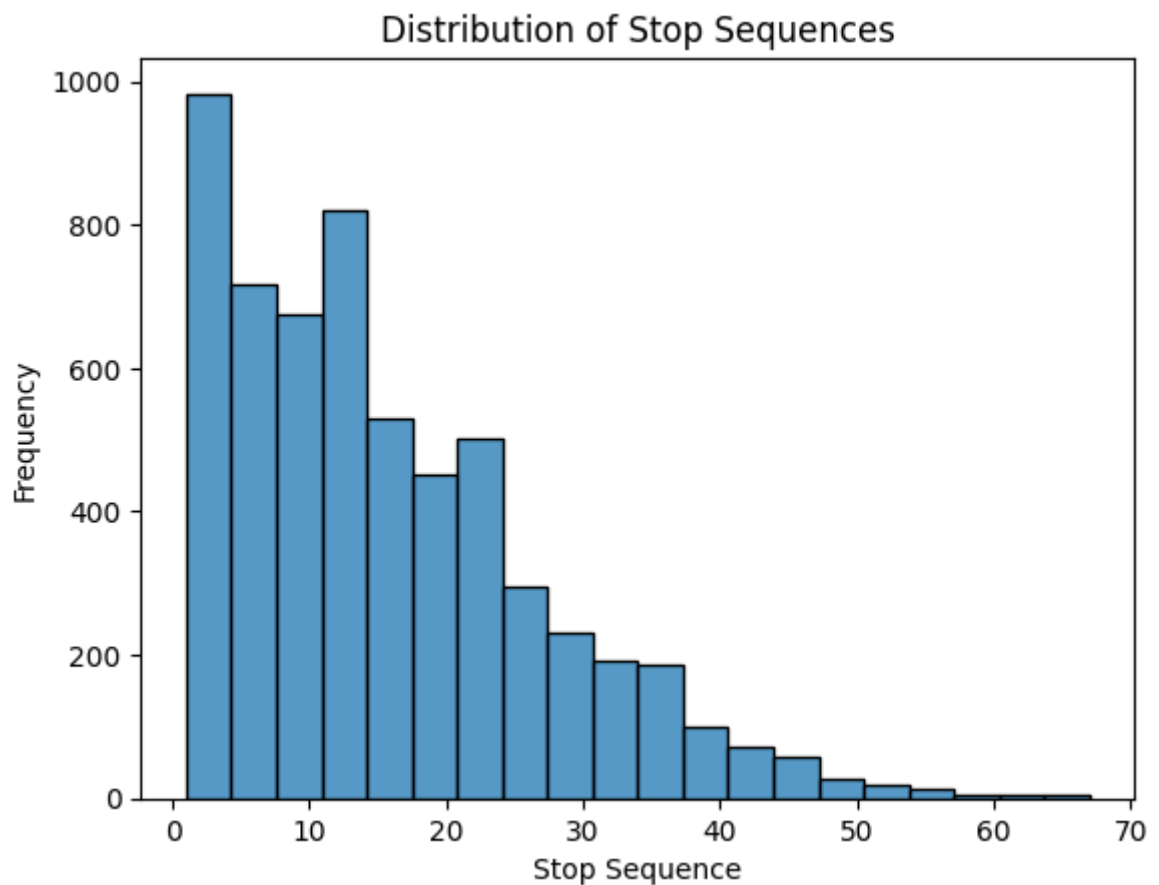


```

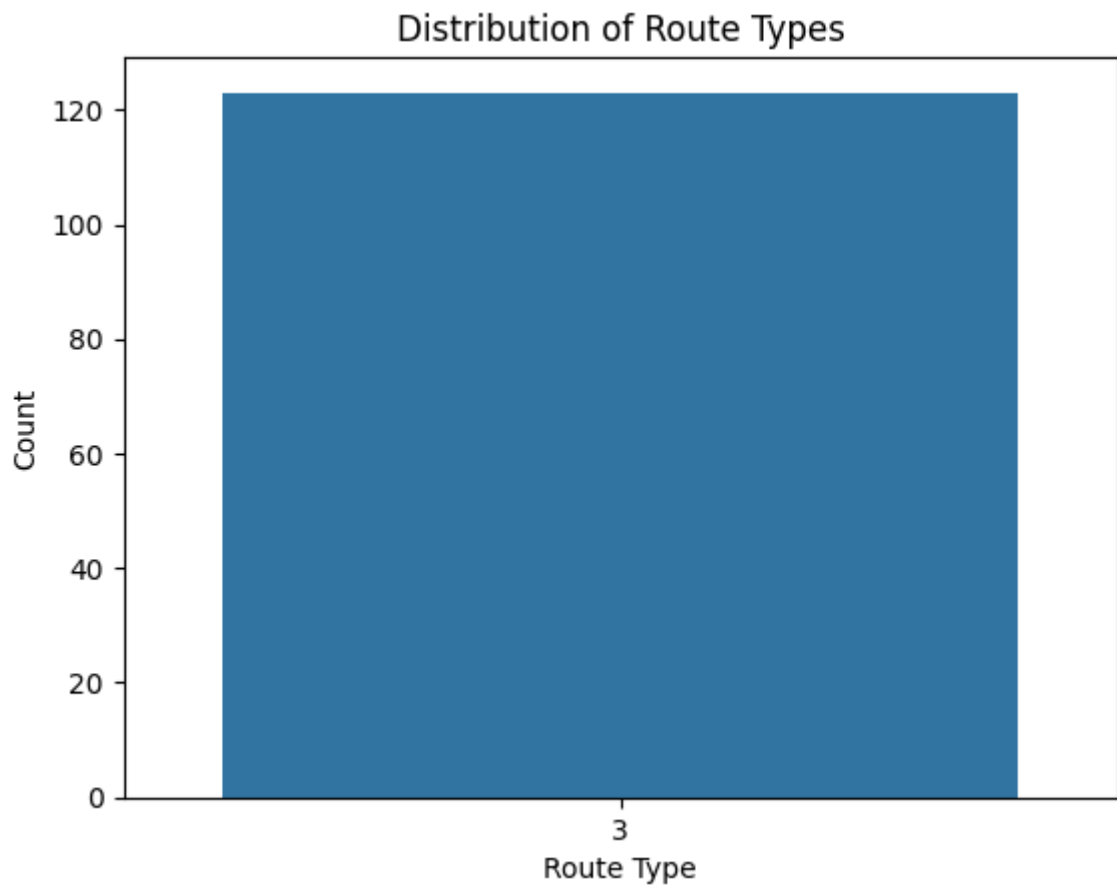
In [ ]: # Plot Histogram
sns.histplot(stop_times['stop_sequence'], bins=20)
plt.xlabel('Stop Sequence')

```

```
plt.ylabel('Frequency')  
plt.title('Distribution of Stop Sequences')  
plt.show()
```



```
In [ ]: # Plot Categorical Plot (Count)  
sns.countplot(x='route_type', data=routes)  
plt.xlabel('Route Type')  
plt.ylabel('Count')  
plt.title('Distribution of Route Types')  
plt.show()
```



```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the stops DataFrame
stops = pd.read_csv(data_folder + 'stops.txt')

# Plot the count of stops by Location type
plt.figure(figsize=(10, 6))
sns.countplot(x='location_type', data=stops)
plt.xlabel('Location Type')
plt.ylabel('Count')
plt.title('Count of Stops by Location Type')
plt.show()
```



```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt

# Define the path to the data folder
data_folder = 'C:/Users/moses_y/OneDrive/Desktop/plots/data/'

# Load the GTFS files into DataFrames
agency = pd.read_csv(data_folder + 'agency.txt')
calendar = pd.read_csv(data_folder + 'calendar.txt')
calendar_dates = pd.read_csv(data_folder + 'calendar_dates.txt')
feed_info = pd.read_csv(data_folder + 'feed_info.txt')
frequencies = pd.read_csv(data_folder + 'frequencies.txt')
routes = pd.read_csv(data_folder + 'routes.txt')
shapes = pd.read_csv(data_folder + 'shapes.txt')
stop_times = pd.read_csv(data_folder + 'stop_times.txt')
stops = pd.read_csv(data_folder + 'stops.txt')
trips = pd.read_csv(data_folder + 'trips.txt')

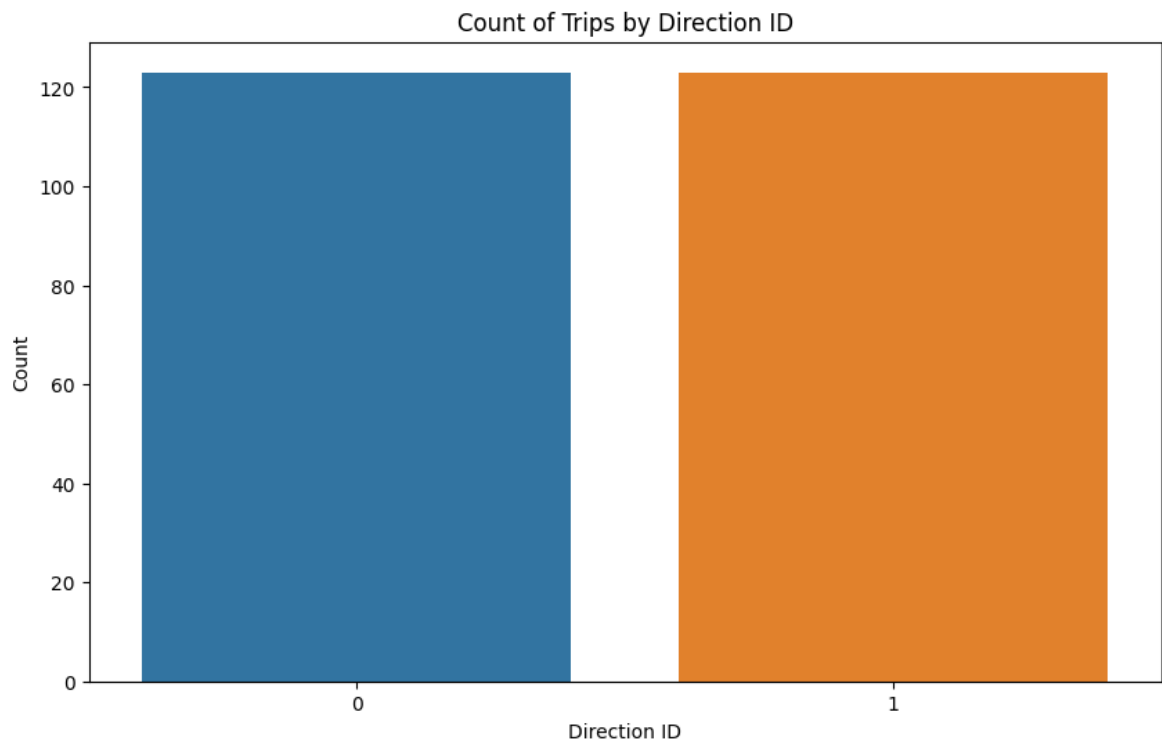
# Extract the 'route_long_name' and 'route_desc' columns from 'routes' DataFrame
route_names = routes['route_long_name']
route_descs = routes['route_desc']

# Plot route names
plt.figure(figsize=(18, 6))
route_names.value_counts().plot(kind='bar')
plt.xlabel('Route Name')
plt.ylabel('Count')
plt.title('Distribution of Route Names')
plt.xticks(rotation=80)
plt.tight_layout()
plt.show()

# Plot route descriptions
plt.figure(figsize=(18, 6))
route_descs.value_counts().plot(kind='bar')
plt.xlabel('Route Description')
```

[illegible][illegible]

11/12



```
In [ ]: import os
os.system('jupyter nbconvert --to pdf --output "C:/Users/moses_y/OneDrive/Desktop/')
```

Out[]: 1

```
In [ ]:
```